

## Tutorial View Code

**Objetivo:** Criar uma tela de login básica com campos de texto para nome de usuário e senha, um botão de login e labels informativas, tudo construído programaticamente em Swift.

## O que é ViewCode?

ViewCode é uma abordagem de desenvolvimento onde você cria interfaces de usuário completamente por código. As principais vantagens incluem:

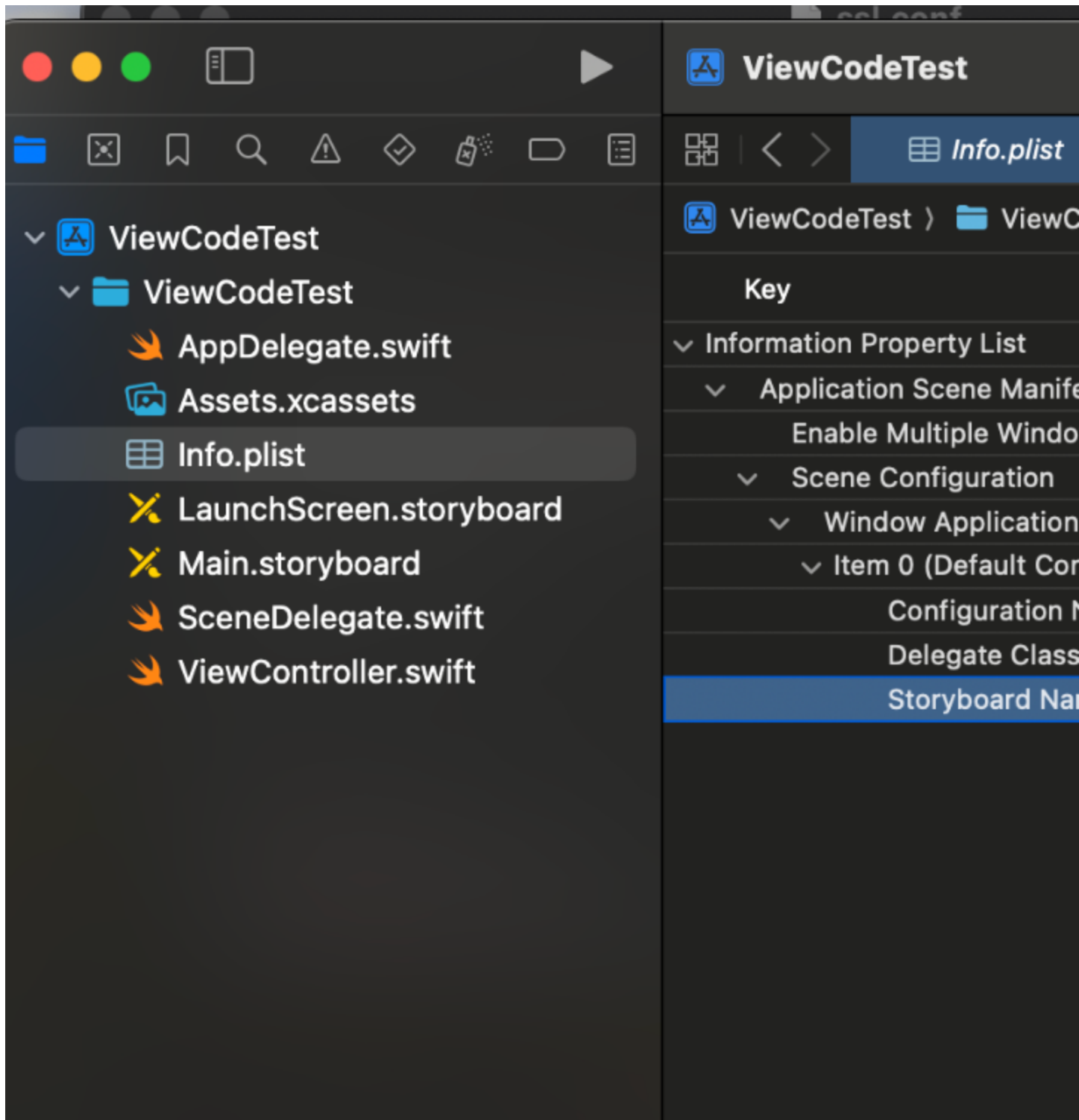
- Melhor controle sobre a UI
- Facilidade para trabalhar em equipe (evita conflitos de merge)
- Mais testabilidade
- Manutenção simplificada
- Melhor performance em projetos grandes

### Passo 1: Criar um Novo Projeto Xcode

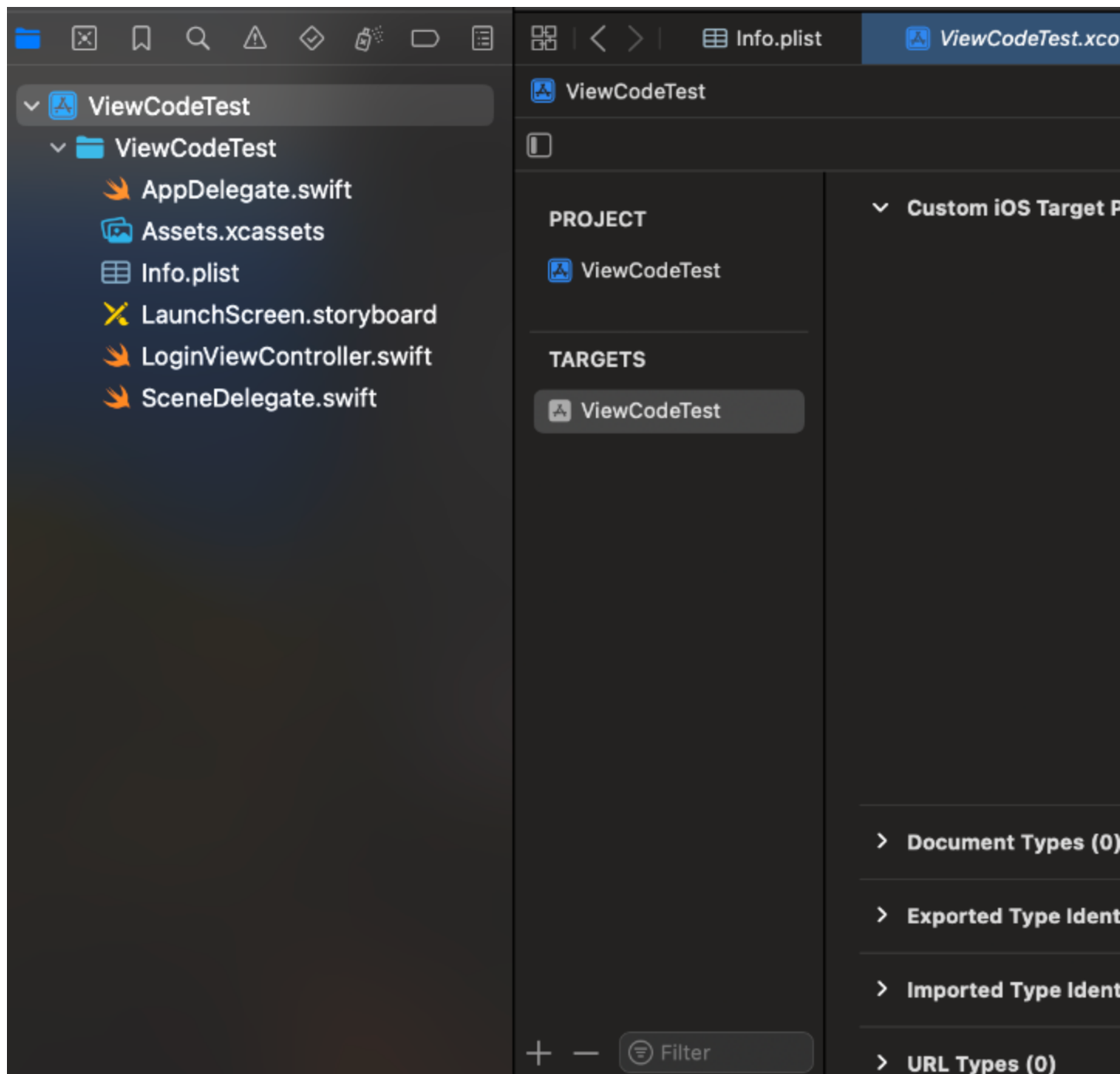
1. Abra o Xcode e selecione "Create a new Xcode project".
2. Escolha "App" na aba iOS e clique em "Next".
3. Configure o projeto:
  - \* Product Name: Nome do seu projeto (ex: "LoginViewCode")
  - \* Interface: **Storyboard** (Precisamos criar o projeto como Storyboard para ele criar a estrutura, vamos remover ele depois)
  - \* Language: Swift
4. Clique em "Next" e escolha onde salvar o projeto.

### Passo 2: Remover Storyboard

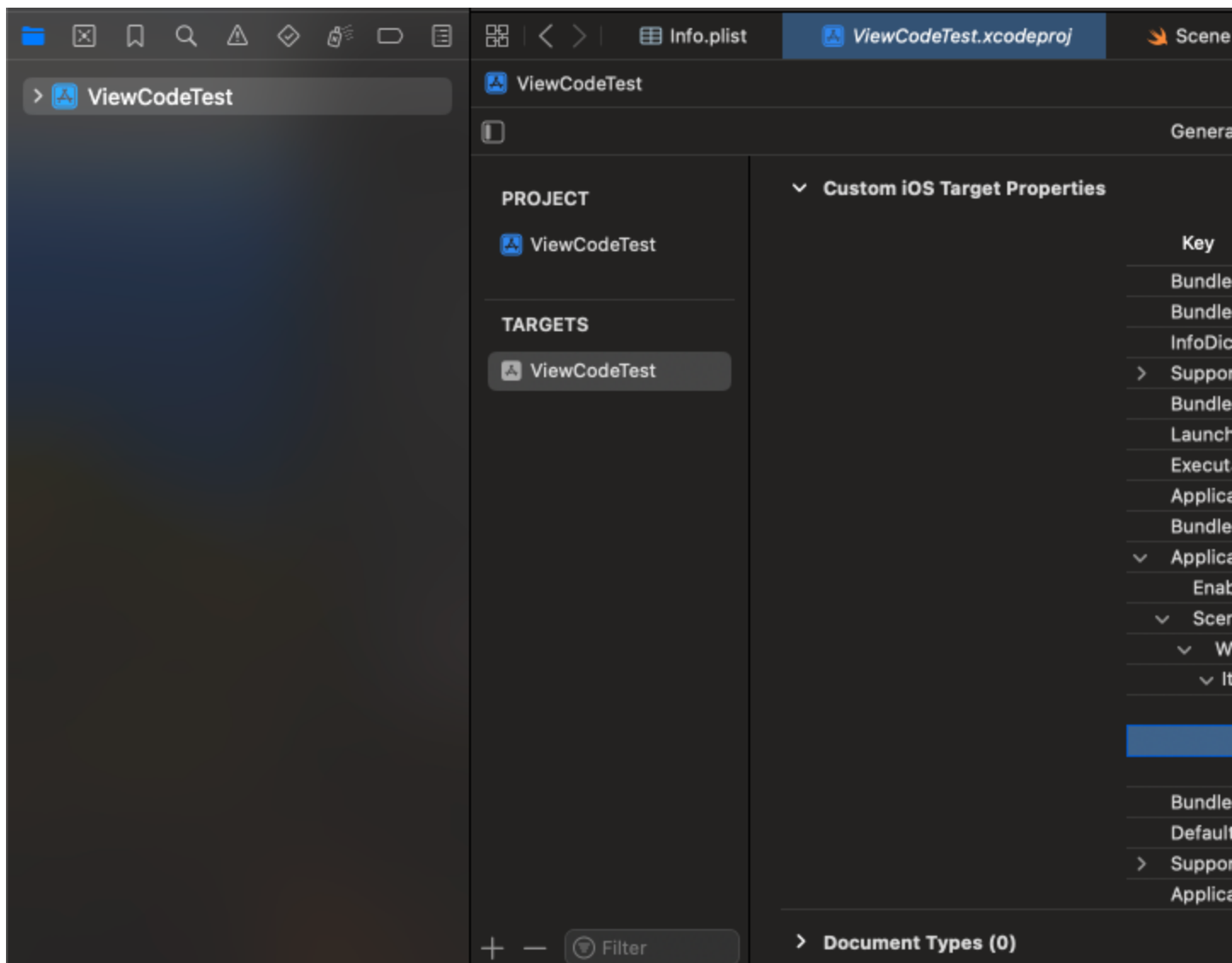
1. Delete o arquivo Main.storyboard e o arquivo ViewController.swift
2. Vá em info.plist, remova a linha "Storyboard Name"



3. Nas propriedades do projeto, que já são abertas logo após a criação do mesmo, precisamos acessar a aba Info e deletar o *Main Storyboard file base name*



4. Nas propriedades do projeto também precisamos deletar o Application Scene Manifest → Scene Configuration → Window Application Session Role → Default Configuration > Storyboard name



5. Vá no arquivo SceneDelegate.swift e altere a função:

```
func scene(_ scene: UIScene, willConnectTo session: UISceneSession,
options connectionOptions: UIScene.ConnectionOptions) {
```

Para:

```
func scene(_ scene: UIScene, willConnectTo session: UISceneSession, options
connectionOptions: UIScene.ConnectionOptions) {

    guard let windowScene = (scene as? UIWindowScene) else { return }

    window = UIWindow(frame: windowScene.coordinateSpace.bounds)
    window?.windowScene = windowScene
    window?.rootViewController = LoginViewController() // Sua ViewController
    window?.makeKeyAndVisible()
```

```
}
```

### Passo 3: Criar a Classe `LoginViewController`

1. File > New > File from template
2. Selecione "Cocoa Touch Class" e clique em "Next".
3. Class: `LoginViewController`
4. Subclass of: `UIViewController`
5. Clique em "Next" e salve o arquivo.

### Passo 4: Declarar os Elementos da Interface

Dentro da classe `LoginViewController`, declare as seguintes propriedades para os elementos da interface:

```
import UIKit

class LoginViewController: UIViewController {

    private let usuarioTextField: UITextField = {
        let textField = UITextField()
        textField.placeholder = "Nome de usuário"
        textField.borderStyle = .roundedRect // Estilo da borda
        textField.translatesAutore sizingMaskIntoConstraints = false // Importante
para Auto Layout
        return textField
    }()

    private let senhaTextField: UITextField = {
        let textField = UITextField()
        textField.placeholder = "Senha"
        textField.borderStyle = .roundedRect
        textField.isSecureTextEntry = true // Esconde a senha
        textField.translatesAutore sizingMaskIntoConstraints = false
        return textField
    }()

    private let loginButton: UIButton = {
        let button = UIButton(type: .system) // Usa o estilo System para o botão
        button.setTitle("Entrar", for: .normal)
        button.backgroundColor = .systemBlue
        button.setTitleColor(.white, for: .normal)
```

```

        button.layer.cornerRadius = 8 // Arredonda as bordas
        button.translatesAutoresizingMaskIntoConstraints = false
        return button
    }()

    private let mensagemLabel: UILabel = {
        let label = UILabel()
        label.text = "Por favor, insira suas credenciais."
        label.textAlignment = .center
        label.numberOfLines = 0 // Permite múltiplas linhas
        label.translatesAutoresizingMaskIntoConstraints = false
        return label
    }()

    override func viewDidLoad() {
        super.viewDidLoad()
        // Configurações iniciais
    }
}

```

### Pontos Importantes:

**`translatesAutoresizingMaskIntoConstraints = false`**: Essa linha é *\*crucial\** para View Code. Por padrão, as views têm um sistema de layout antigo habilitado. Desabilitar isso permite que você use Auto Layout programaticamente (constraints).

### Passo 5: Adicionar as Subviews e Configurar o Layout

1. Dentro do método `viewDidLoad()`, configure a cor de fundo da view principal e adicione as subviews:

```

override func viewDidLoad() {
    super.viewDidLoad()
    view.backgroundColor = .white // Cor de fundo

    // Adiciona as subviews à view principal
    view.addSubview(usuarioTextField)
    view.addSubview(senhaTextField)
    view.addSubview(loginButton)
    view.addSubview(mensagemLabel)

    // Configurar as constraints (Auto Layout)
}

```

```
    configurarConstraints()  
}
```

2. Crie uma função `configurarConstraints()` para definir as constraints usando Auto Layout:

```
func configurarConstraints() {  
    // Constraints para o usuarioTextField  
    NSLayoutConstraint.activate([  
        usuarioTextField.centerXAnchor.constraint(equalTo: view.centerXAnchor), //  
        Centralizado horizontalmente  
        usuarioTextField.topAnchor.constraint(equalTo:  
        view.safeAreaLayoutGuide.topAnchor, constant: 50), // Distância do topo  
        usuarioTextField.widthAnchor.constraint(equalTo: view.widthAnchor,  
        multiplier: 0.8), // Largura (80% da tela)  
        usuarioTextField.heightAnchor.constraint(equalToConstant: 40) // Altura fixa  
    ])  
  
    // Constraints para o senhaTextField  
    NSLayoutConstraint.activate([  
        senhaTextField.centerXAnchor.constraint(equalTo: view.centerXAnchor),  
        senhaTextField.topAnchor.constraint(equalTo: usuarioTextField.bottomAnchor,  
        constant: 20),  
        senhaTextField.widthAnchor.constraint(equalTo: view.widthAnchor, multiplier:  
        0.8),  
        senhaTextField.heightAnchor.constraint(equalToConstant: 40)  
    ])  
  
    // Constraints para o loginButton  
    NSLayoutConstraint.activate([  
        loginButton.centerXAnchor.constraint(equalTo: view.centerXAnchor),  
        loginButton.topAnchor.constraint(equalTo: senhaTextField.bottomAnchor,  
        constant: 30),  
        loginButton.widthAnchor.constraint(equalTo: view.widthAnchor, multiplier:  
        0.5),  
        loginButton.heightAnchor.constraint(equalToConstant: 40)  
    ])  
  
    // Constraints para o mensagemLabel  
    NSLayoutConstraint.activate([  
        mensagemLabel.centerXAnchor.constraint(equalTo: view.centerXAnchor),  
        mensagemLabel.topAnchor.constraint(equalTo: loginButton.bottomAnchor,  
        constant: 20),  
    ])  
}
```

```

        mensagemLabel.leadingAnchor.constraint(equalTo: view.leadingAnchor,
        constant: 20),
        mensagemLabel.trailingAnchor.constraint(equalTo: view.trailingAnchor,
        constant: 20) // Margens laterais
    ])
}

```

## Entendendo as Constraints:

- \* `centerXAnchor.constraint(equalTo: view.centerXAnchor)`: Centraliza o elemento horizontalmente na view principal.
- \* `topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor, constant: 50)`: Posiciona o elemento a 50 pontos da parte superior da "safe area" (região visível da tela, levando em conta a barra de status, notch, etc.).
- \* `widthAnchor.constraint(equalTo: view.widthAnchor, multiplier: 0.8)`: Define a largura como 80% da largura da view principal.
- \* `heightAnchor.constraint(equalToConstant: 40)`: Define uma altura fixa de 40 pontos.
- \* `leadingAnchor.constraint(equalTo: view.leadingAnchor, constant: 20)` e `trailingAnchor.constraint(equalTo: view.trailingAnchor, constant: 20)`: define as margens laterais

## Passo 6: Adicionar Funcionalidade ao Botão

1. Adicione um target (ação) ao botão para lidar com o toque:

```

override func viewDidLoad() {
    super.viewDidLoad()
    view.backgroundColor = .white
    // ... (seu código existente) ...

    loginButton.addTarget(self, action: #selector(loginButtonTapped), for:
    .touchUpInside)
}

@objc func loginButtonTapped() {
    print("Botão de login tocado!")
    // Aqui você colocaria a lógica de autenticação
}

```

## Passo 7: Rodar o Aplicativo



1. Selecione um simulador ou dispositivo físico no canto superior esquerdo do Xcode.
2. Clique no botão "Run" (Play).

Você deverá ver a tela de login construída completamente em código, sem o uso de Storyboard.

### Pontos Chave:

- \* `translatesAutoresizingMaskIntoConstraints = false`: Lembre-se de sempre desabilitar isso para usar Auto Layout programaticamente.
- \* `safeAreaLayoutGuide`: Use a safe area para evitar sobreposição com a barra de status ou outros elementos da interface.
- \* `NSLayoutConstraint.activate()`: Ativa as constraints para que o Auto Layout funcione.
- \* **Convenience**: O método `configurarConstraints()` ajuda a manter o código mais organizado e legível.